

Software Engineering Lab-3

Name: SAGAR A WALIKAR

SRN: PES1UG24CS396

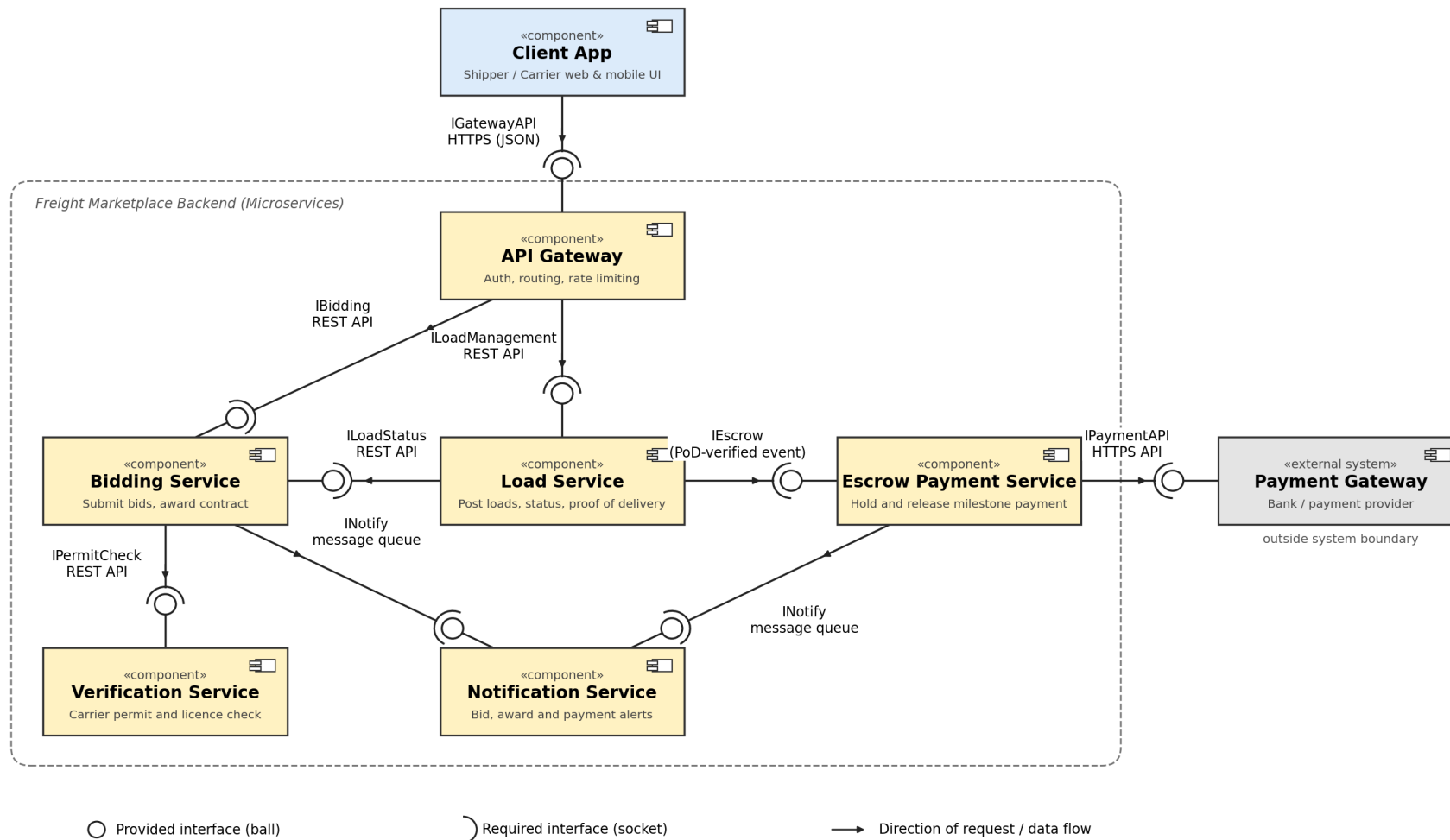
GitHub Link: <https://github.com/sagarwalikar999/SE-Labs-PES1UG24CS396>

Date: 28-09-2026

SEC: 5_G

Component Diagram

Component Diagram: Inter-City Freight Load Matching Marketplace (Microservices)



Architectural Justification

Inter-City Freight Load Matching Marketplace | PES1UG24CS396 | SEC 5_G

We chose Microservices Architecture for the Inter-City Freight Load Matching Marketplace System.

Reason 1: Bidding has to scale on its own. NFR-002 asks for at least 500 concurrent bidding sessions with a bid response time under 2 seconds. Bidding traffic comes in bursts (for example end-of-quarter shipping rushes), while posting loads or releasing payments happens far less often. With microservices we can run more copies of the Bidding Service when it is busy without scaling the Load, Escrow or Notification services. In a layered or client-server design the whole application would have to be scaled together.

Reason 2: The system is made of separate, loosely related functions. Posting loads, bidding, permit verification, escrow payment and notifications each have their own job and their own interface (IBidding, IPermitCheck, IEscrow, INotify and so on in the diagram). If the Notification Service goes down, carriers can still bid and shippers can still award loads. The Verification Service can also be changed (for example to use a new permit database) without touching Bidding, as long as IPermitCheck stays the same.

Security advantage: Only the Escrow Payment Service talks to the external Payment Gateway and handles payment data, so a problem in another service does not expose payment details. All outside traffic from shippers and carriers enters through the API Gateway, which does authentication in one place. This supports the security part of NFR-001 (secure escrow release) and keeps unverified users away from the internal services.

Performance benefit: Escrow release is triggered by an event (proof of delivery verified) instead of waiting for a user request, so the payout can start immediately and not be delayed by other work. This helps meet the 60-second payout target in NFR-001. Notifications are also sent through a message queue, so bidding does not have to wait for alerts to be delivered.

Trade-off we accept: Microservices add network calls between services and make data consistency harder. The main risk is awarding one load to two carriers (FR-003), so the award step is kept inside the Bidding Service as a single atomic operation. We considered Layered and Client-Server, but a single scalable unit or a single central server would be a bottleneck and a single point of failure for bidding and payments.